

รายงาน

Elliptic Curve Cryptography

จัดทำโดย

นาย ประวิน หมวดสุข	6834432623
นาย ปัญญากร อินทะสมบัติ	6834433223
นาย ศุภกรณ์ คำวงษา	6834456723
นาย สรวิชญ์ นวมวัตร์	6834458023

เสนอ

ผศ.ดร. วุฒิชัย จงจิตเมตต์

รายงานนี้เป็นส่วนหนึ่งของรายวิชา 2301230 Discrete Math for CS

สาขาวิทยาการคอมพิวเตอร์ คณะวิทยาศาสตร์

จุฬาลงกรณ์มหาวิทยาลัย

คำนำ

Public-key cryptography ถูกเสนอโดย Diffie และ Hellman ในปี 1976[1] ระบบการถอดรหัสช่วยให้ทั้งสองฝ่ายที่ไม่ได้พบกันสามารถติดต่อกันได้ สามารถตกลงในการใช้ Secret Key หรือตรวจสอบ ข้อความผ่าน Open Network ความปลอดภัยของระบบนี้ขึ้นอยู่กับปัญหาทางคณิตศาสตร์ง่ายๆ แต่ แก้ย้อนกลับได้ยาก ระบบแรกๆที่นำมาใช้งานอย่างแพร่หลายคือ **Diffie-Hellman Key Exchange** และ **RSA**[2] ซึ่งใช้การคำนวณแบบ Arithmetic modulo ด้วยจำนวนเต็มที่ใหญ่มาก ความปลอดภัยทั้ง สองระบบนี้ผูกติดอยู่กับความยากของ Discrete Logarithm ใน $\mathbb{Z}_p^*$ ย่อมาจากคำว่า **Multiplicative Group of Integers Modulo** p และปัญหาแยกตัวประกอบจำนวนเต็มตามลำดับ

ในปี 1958 มี Neal Koblitz และ Victor Miller ได้เสนอแนวคิด ในการเปลี่ยนจากการใช้ Multiplicative Group ของ Finite Field มาใช้ Group of Points บน Elliptic Curve แทน ข้อสังเกตสำคัญคืออัลกอรีทึมที่เท่าที่รู้จักในการแก้ปัญหา Discrete Logarithm ในกลุ่มจุดบน Elliptic Cueve[3][4] ทำงานได้ช้ากว่าอัลกอลิทึมที่ใช้แก้ปัญหาใน $\mathbb{Z}_p^*$ (Multiplicative Group of Integers Modulo p) หรือแยกตัวประกอบมาก ด้วยเหตุนี้ **ECC (Elliptic Curve Cryptography)** จึงเข้ามาแก้ปัญหา และสามารถยกระดับความปลอดภัยได้ในระดับที่เท่ากันโดยใช้ขนาดคีย์ (Key Size) ที่เล็กกว่ามาก คีย์ของ Elliptic Curve ขนาด 256 bit ให้ครอบคุมพอๆ กับคีย์ RSA ที่มีขนาด bit ถึง 3072 bit

ขนาด bit ที่เล็กลงหมายถึงการใช้ Bandwidth และพื้นที่จัดเก็บน้อยลง รวมถึงประมวลผลได้เร็วขึ้น สิ่งเหล่านี้สำคัญมากสำหรับการใช้งานบนอุปกรณ์ที่มีพื้นที่อย่างจำกัด เช่น สมาร์ตการ์ด, Embedded Controllers และโทรศัพท์ รวมถึงฝั่งเซิร์ฟเวอร์ แม้ในช่วงแรกจะมีความช้า แต่ ECC ก็ได้กลายมาเป็นตัวเลือกมาตรฐานในช่วง 2010 ในปัจจุบัน การเข้ารหัส **X25519** เป็น Key Exchange ที่ถูกนำมาใช้เจรจามากที่สุดใน TLS 1.3[5] ในขณะที่ **Ed25519** ประเภทคีย์ที่แนะนำให้ใช้สำหรับ Secure Shell (SSH) ก็ถูกใช้ปกป้องระบบ Cryptocurrency ที่มีมูลค่าหลายแสนล้านล้าน

เนื้อหาในส่วนฉบับนี้ถูกจัดระเบียบไว้ดังนี้ **ส่วนที่ 1** จะแนะนำคณิตศาสตร์ของ Elliptic Curves **ส่วนที่ 2** อธิบายเกี่ยวกับ Hardness assumption และการโจมตี **ส่วนที่ 3** นำเสนอ Protocals **ส่วนที่ 4** เปรียบเทียบ Standardized Curves **ส่วนที่ 5** ครอบคลุมเรื่องความปลอดภัย **ส่วนที่ 6** ตรวจสอบภัยคุกคามจากคอมพิวเตอร์ควอนตัม **ส่วนที่ 7** บทสรุป

สารบรรณ

	หน้า
คำนำ	ก
สารบรรณ	ข
1. Mathematical Background	0
1.1. Finite Fields	0
1.2. Elliptic Curves	0
1.3. The Group Law	1
1.4. Group order and subgroups	2
1.5. Scalar multiplication	2
1.6. A worked example over $\mathbb{F}_{17}$	3
2. The Elliptic Curve Discrete Logarithm Problem	4
2.1. Generic Attacks	5
2.2. Weak Curves	6
3. Cryptographic Protocols	7
3.1. Elliptic Curve Diffie–Hellman (ECDH)	7
3.2. Elliptic Curve Digital Signature Algorithm (ECDSA)	8
3.3. EdDSA and Ed25519	9
3.4. Elliptic Curve Integrated Encryption Scheme (ECIES)	9
4. Standardized Curves	10
4.1. รูปแบบของเส้นโค้ง	10
4.2. ความเชื่อมั่นในการสร้างเส้นโค้ง	11
5. Implementation Security	11
6. The Quantum Threat and Post-Quantum Migration	13
7. Conclusion	14
References	15

1. Mathematical Background

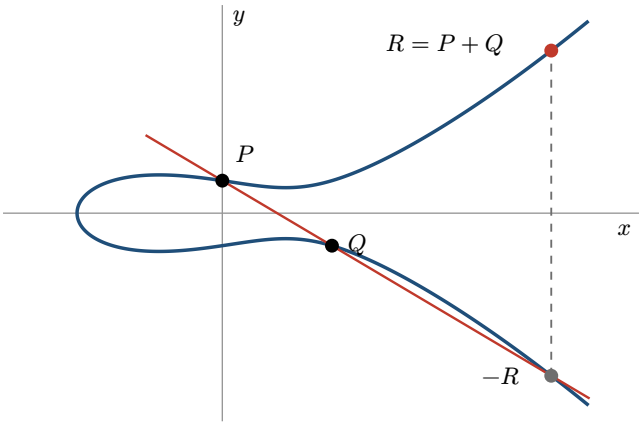
1.1. Finite Fields

การเข้ารหัสแบบวงรีถูกกำหนดโดย finite field $\mathbb{F}_q$ โดยในทางปฏิบัติ ฟิลด์ดังกลาวเกือบทั้งหมดจะเป็นจำนวนเฉพาะ $\mathbb{F}_p = \{0, 1, ..., p-1\}$ ซึ่งมีการดำเนินการทางคณิตศาสตร์แบบ modulo ด้วยจำนวนเฉพาะขนาดใหญ่ p ส่วนฟิลด์ฐานสอง $\mathbb{F}_{2^m}$ นั้นเคยได้รับความนิยมในการนำไปใช้งานบนฮาร์ดแวร์ แต่มาตรฐานสมัยใหม่ส่วนใหญ่ได้เลิกสนับสนุนการใช้งานฟิลด์ประเภทนี้ไปแล้ว[6] ทั้งนี้สมาชิกทุกตัวที่ไม่ใช่ศูนย์ใน $\mathbb{F}_p$ จะมีตัวผกผันสำหรับการคูณซึ่งสามารถคำนวณได้โดยใช้อัลกอรีทึม Extended Euclidean หรือคำนวณจาก $a^{-1} = a^{p-2} \bmod p$ ตามทฤษฎีบท Fermat's little theorem

1.2. Elliptic Curves

> **นิยามที่ 1** (**เส้นโค้งวงรี**). ให้ $p > 3$ เป็นจำนวนเฉพาะ เส้นโค้งวงรี E บนฟิลด์ $\mathbb{F}_p$ ในรูปแบบไวเออร์สตราสแบบย่อ (short Weierstrass form) คือเซตของคำตอบ $(x, y) \in \mathbb{F}_p^2$ ของสมการ
>
> $$y^2 = x^3 + ax + b, \quad a, b \in \mathbb{F}_p \tag{1}$$
>
> รวมถึงจุดพิเศษที่จุด*อนันต์* $\mathcal{O}$ โดยเส้นโค้งดังกล่าวจะต้องไม่มีจุดเอกฐาน กล่าวคือ ค่าดิสคริมิแนนต์ของเส้นโค้งต้องเป็นไปตามเงื่อนไข $4a^3 + 27b^2 \neq 0$.

เงื่อนไขการไม่มีจุดเอกฐาน รับประกันว่าเส้นโค้งจะไม่มีจุดแหลมหรือจุดตัดตัวเอง ดังนั้นจึงมีเส้นสัมผัสเพียงเส้นเดียวที่ทุกๆจุด บนเซตของจำนวนจริงสามารถวาดเส้นโค้งนี้ได้ และภาพนี้เองที่เป็นที่มาของกฎกลุ่ม (รูปที่ 1) ส่วนบนฟิลด์จำกัด $\mathbb{F}_p$ นั้น "เส้นโค้ง" จะเป็นเซตของจุดที่กระจัดกระจาย (รูปที่ 2) แต่ก็ยังคงใช้สูตรพีชคณิตเดียวกันได้เสมอ



รูปที่ 1: กฎคอร์ดและเส้นสัมผัส บนเส้นโค้ง $y^2 = x^3 - x + 1$ เหนือฟิลด์จำนวนจริง $\mathbb{R}$: เส้นตรงที่ลากผ่านจุด $P = (0, 1)$ และ $Q = (1, -1)$ จะตัดกับเส้นโค้งเป็นครั้งที่สามที่จุด $-R = (3, -5)$ และเมื่อสะท้อนข้ามแกน x จะได้จุดผลรวมเป็น $R = P + Q = (3, 5)$

1.3. The Group Law

จุดต่าง ๆ บนเส้นโค้ง E ร่วมกับ $\mathcal{O}$ ก่อตัวเป็น *อาบีเลียนกรุป (abelian group)* ภายใต้การดำเนินการที่เขียนในรูปแบบการบวก ในทางเรขาคณิต การบวกจุดสองจุดคือ P และ Q ทำได้โดยการลากเส้นตรงผ่านจุดทั้งสอง เส้นตรงนี้จะตัดเส้นโค้งที่จุดอื่นอีกเพียงหนึ่งจุดเสมอ และภาพสะท้อนข้ามแกน x ของจุดนั้นก็คือ $P + Q$ สำหรับการบวกจุดเดิมซ้ำสองเท่า ให้ใช้เส้นสัมผัสแทน จุดที่อนันต์ทำหน้าที่เป็นเอกลักษณ์: $P + \mathcal{O} = P$ และอินเวิร์สของ $P = (x, y)$ คือ $-P = (x, -y)$

เมื่อแปลงเรขาคณิตให้อยู่ในรูปพีชคณิต จะได้สูตรที่ชัดเจนดังนี้ กำหนดให้ $P = (x_1, y_1)$ และ $Q = (x_2, y_2)$ โดยที่ $P \neq -Q$ จะได้ $P + Q = (x_3, y_3)$ โดยที่

$$\lambda = \begin{cases} \frac{y_2 - y_1}{x_2 - x_1} \text{ if } P \neq Q & \text{(addition)} \\ \frac{3x_1^2 + a}{2y_1} \text{ if } P = Q & \text{(doubling)} \end{cases} \tag{2}$$

$$x_3 = \lambda^2 - x_1 - x_2, \qquad y_3 = \lambda(x_1 - x_3) - y_1. \tag{3}$$

การดำเนินการทั้งหมดกระทำบน $\mathbb{F}_p$ ดังนั้นการหารใน Equation 2 จึงหมายถึงการคูณด้วยอินเวิร์สการคูณแบบมอดุลาร์หาก $P = -Q$ เส้นตรงที่ลากผ่านจะเป็นเส้นแนวตั้ง และจะได้ $P + Q = \mathcal{O}$ ซึ่งสามารถตรวจสอบได้ว่าการดำเนินการนี้มีสมบัติการเปลี่ยนหมู่และการสลับที่ จึงทำให้ $(E(\mathbb{F}_p), +)$ เป็นอาบีเลียนกรุปจำกัด[7]

1.4. Group order and subgroups

ทฤษฎีบทของฮัสเซอ (Hasse's theorem) กำหนดขอบเขตของจำนวนจุดบนเส้นโค้งเหนือ $\mathbb{F}_p$:

$$\left|\left|E(\mathbb{F}_p)\right| - (p+1)\right| \leq 2\sqrt{p}. \tag{4}$$

ดังนั้น เส้นโค้งเหนือฟิลด์ขนาด 256 บิต จะมีจุดอยู่ประมาณ 2^{256} จุด จำนวนเต็ม $t = p + 1 - \left|E(\mathbb{F}_p)\right|$ เรียกว่า *รอยทางของโฟรเบนิอุส (trace of Frobenius)* โดยการนับจำนวนจุดสามารถคำนวณได้อย่างมีประสิทธิภาพด้วยขั้นตอนวิธีของชูฟ (Schoof's algorithm) และขั้นตอนวิธีที่พัฒนาต่อยอด

ระบบวิทยาการรหัสลับมักทำงานบนกรุปย่อยแบบวัฏจักรที่มีอันดับเป็นจำนวนเฉพาะขนาดใหญ่ พารามิเตอร์โดเมนมาตรฐาน (*domain parameters*) ประกอบด้วยทูเพิล (p, a, b, G, n, h) ในที่นี้ G คือ *จุดฐาน (base point)* หรือตัวก่อกำเนิดที่มีอันดับเฉพาะ n ซึ่งหมายถึง $nG = \mathcal{O}$ ส่วน *โคแฟกเตอร์ (cofactor)* คือ $h = \left|E(\mathbb{F}_p)\right|/n$ ซึ่งในอุดมคติควรมีค่าเป็น 1 และในทางปฏิบัติจะมีค่าน้อย (เช่น $h = 8$ สำหรับ Curve25519)

1.5. Scalar multiplication

การดำเนินการหลักของ ECC คือการคูณสเกลาร์ กล่าวคือ เมื่อกำหนดจำนวนเต็ม k และจุด P ให้คำนวณ

$$kP = \underbrace{P + P + \ldots + P}_{k \text{ ครั้ง}} \tag{5}$$

การคำนวณด้วยวิธีที่ตรงไปตรงมาจะต้องอาศัยการบวกจำนวน $k-1$ ครั้ง ซึ่งเป็นไปไม่ได้ในทางปฏิบัติเมื่อ $k \approx 2^{256}$ ส่วนวิธี *double-and-add* (อัลกอริทึมที่ 1) จะประมวณผลจากการกระจายค่า k ในรูปแบบเลขฐานสอง ซึ่งอาศัยการดำเนินงานแบบดับเบิล ประมาณ $\log_2 k$ ครั้ง และการบวกอีกเพียงครึ่งหนึ่งของจำนวนดังกล่าวเท่านั้น

อัลกอริทึมที่ 1: การคูณสเกลาร์ด้วยวิธีดับเบิลแอนด์แอด (double-and-add) แบบจากซ้ายไปขวา

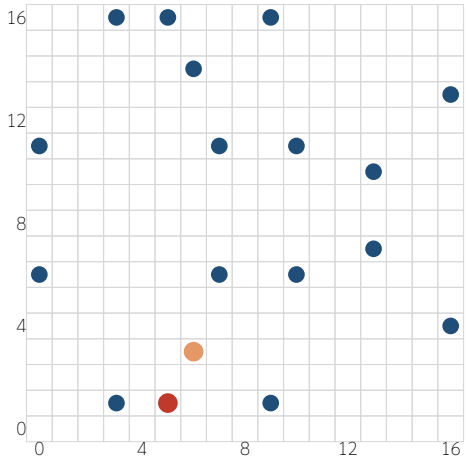
Input: จุด P, สเกลาร์ $k = (k_{m-1} \ldots k_1 k_0)_2$ โดย $k_{m-1} = 1$
Output: $Q = kP$
1. $Q \leftarrow P$
2. สำหรับ $i = m-2$ ลดลงจนถึง 0 ให้ทำดังนี้
   a. $Q \leftarrow 2Q$ (doubling)
   b. ถ้า $k_i = 1$ ให้ $Q \leftarrow P + Q$ (addition)
3. คืนค่า Q

การแตกแขนงตามค่า k_i ส่งผลให้เวลาที่ใช้ในการประมวลผลและการใช้พลังงานขึ้นอยู่กับบิตลับค่า k ดังนั้นในโค้ดที่นำไปใช้งานจริงจึงเลือกใช้วิธีที่ใช้เวลาคงที่ เช่นบันไดมอนต์โกเมอรี (Montgomery ladder)[8] ซึ่งจะดำเนินการบวกหนึ่งครั้งและดำเนินการเพิ่มค่าเป็นสองเท่า หนึ่งครั้งสำหรับทุกบิต โดยไม่คำนึงว่าบิตนั้นจะมีค่าเป็นเท่าใด

1.6. A worked example over $\mathbb{F}_{17}$

เพื่อให้เห็นภาพว่าคณิตศาสตร์ของ Elliptic Curve ทำงานยังไง ลองพิจารณาตัวอย่างการคำนวณสมการเส้นโค้ง $E : y^2 = x^3 + 2x + 2$ ที่ทำงานอยู่บนฟิลด์ $\mathbb{F}_{17}$

- ตรวจสอบความใช้ได้ของเส้นโค้ง ต้องคำนวณค่า Discriminant ของสมการ ซึ่งเมื่อคิดเลขออกมาแล้วมีค่าเท่ากับ $4 \pmod{17}$ เนื่องจากค่านี้ไม่ได้เป็น 0 เป็นการยืนยันว่าเส้นโค้ง E นี้เป็นแบบ Non-singular สามารถนำเข้ารหัสได้



รูปที่ 2: จุดทั้ง 18 จุดของ $y^2 = x^3 + 2x + 2$ เหนือฟิลด์ $\mathbb{F}_{17}$ บนตารางขนาด 17×17 ของ $\mathbb{F}_{17}^2$ จุดฐาน $G = (5, 1)$ แสดงด้วยสีแดง และ $2G = (6, 3)$ แสดงด้วยสีส้ม

- การเลือกจุดเริ่มต้น 19 เป็นจำนวนเฉพาะ กฎคณิตศาสตร์บอกว่ากลุ่มตัวเลขนี้ทำงานลูป สามารถหยิบช่องไหนมาก็ได้ใน 18 ช่องเป็น “ช่องเริ่ม” เพื่อใช้สร้างช่องที่เหลือ เราเลือกช่อง $G = (5, 1)$ เป็นช่องเริ่ม (ช่องสีแดง)
- สมมาตรของกราฟ จากกราฟในรูปที่นำจุดทั้ง 18 จุดมาพล็อตบนตารางขนาด 17×17 จะเห็นว่าจุดต่างๆ มีความสมมาตร พับครึ่งกันได้พอดีที่เส้นแกนกลาง $y = 8.5$ นี่เป็นผลมาจากคุณสมบัติของ Modulo 17 ที่ค่า $-(x, y)$ จะมีค่าเท่ากับ $(x, 17 - y)$ เสมอ

การคำนวณหาจุดถัดไป (Doubling)

หากเราต้องการหาจุด $2G$ (ซึ่งคือการนำจุด G มาบวกกับตัวเอง) เราจะต้องใช้สูตรทางคณิตศาสตร์เรียกว่า Doubling

1. เริ่มจากหาความชันของเส้นสัมผัส โดยแทนค่าเข้าไปในสมการคณิตศาสตร์ ซึ่งผลลัพธ์สุดท้ายจะคำนวณออกมาได้ $\lambda \equiv 13 \pmod{17}$
2. นำความชันที่ได้ไปคำนวณหาพิกัดใหม่ (x_3, y_3) ซึ่งจะได้ $x_3 \equiv 6$ และ $y_3 \equiv 3 \pmod{17}$
3. ดังนั้น เราจึงได้ $2G = (6, 3)$ (ช่องสีส้มในกราฟ)

วงจรการทำงานแบบสมบูรณ์ (Full cycle)

หากเราต้องการทำกระบวนการนี้ซ้ำไปเรื่อยๆ (เป็น $3G$, $4G$, $5G$, ...) เราจะได้พิกัดใหม่เรื่อยๆ ตามตารางที่ 1 ข้อสังเกตที่น่าสนใจคือ

ตารางที่ 1: ผลคูณของ $G = (5, 1)$ บน $y^2 = x^3 + 2x + 2$ เหนือฟิลด์ $\mathbb{F}_{17}$ และสุดท้าย $19G = \mathcal{O}$

k	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**
kG	(5,1)	(6,3)	(10,6)	(3,1)	(9,16)	(16,13)	(0,6)	(13,7)	(7,6)
k	**10**	**11**	**12**	**13**	**14**	**15**	**16**	**17**	**18**
kG	(7,11)	(13,10)	(0,11)	(16,4)	(9,1)	(3,16)	(10,11)	(6,14)	(5,16)

พิกัดในตารางครึ่งหลัง จะเป็นภาพสะท้อนของตารางครึ่งแรกพอดีตามสมการ $kG = -(19 - k)G$ เมื่อเราบวกจุดไปเรื่อยๆ ผลลัพธ์จะหลุดเส้นและวนกลับไป “จุดอนันต์” $\mathcal{O}$ พอดี ซึ่งถือเป็นการจบลูป

2. The Elliptic Curve Discrete Logarithm Problem

> **นิยามที่ 2** (ECDLP). กำหนดให้มีพารามิเตอร์โดเมนพร้อมด้วยจุดฐาน G เป็นอันดับจำนวนเฉพาะ n และจุด $Q \in \langle G \rangle$ จงหาจำนวนเต็ม $k \in [0, n-1]$ ที่ทำให้ $Q = kG$

การคูณสเกลาร์เป็นฟังก์ชั่นทางเดียว การคำนวน $Q = kG$ ใช้เวลาเพียงระดับไมโครวินาที แต่การหาค่า k คืนจาก Q ดูเหมือนจะเป็นไปไม่ได้ในทางปฏิบัติหากเลือกเส้นโค้งไว้อย่างอย่างเหมาะสม ในตัวอย่างจำลองการหาค่า k ที่ทำให้ $kG = (16, 4)$ นั้นทำได้ง่าย เพราะอ่านได้จากตารางที่ 1 ว่า $k = 13$ แต่เมื่อ $n \approx 2^{256}$ จะไม่สามารถสร้างตารางหรือใช้วิธีค้นหาแบบไล่เลียงทุกกรณีได้เลย

2.1. Generic Attacks

อัลกอริทึมที่รู้จักกันดีที่สุดสำหรับ ECDLP บนเส้นโค้งที่เลือกไว้อย่างดีนั้นเป็นแบบทั่วไป พวกมันใช้เพียงการดำเนินการกลุ่มและจะทำงานได้ในกลุ่มใดก็ได้ วิธีที่ใช้งานได้จริงที่สุดคือวิธี rho ของ Pollard[9] ซึ่งหาการชนกัน ในการเดินสุ่มเทียม ในเวลาประมาณ

$$\sqrt{\frac{\pi n}{4}} \approx 0.886\sqrt{n} \tag{6}$$

การดำเนินการในกลุ่ม ใช้หน่วยความจำน้อยมาก วิธีการขนานแบบ Van Oorschot–Wiener ช่วยให้ความเร็วเพิ่มขึ้นเป็นสัดส่วนเชิงเส้นตามจำนวนหน่วยประมวลผล ส่วนอัลกอริทึม Pohlig–Hellman จะลดทอนปัญหาลงเหลือเพียงการพิจารณาตัวประกอบเฉพาะที่มีค่ามากที่สุดของ n ด้วยเหตุนี้ n จึงจำเป็นต้องเป็นจำนวนเฉพาะที่มีค่ามาก โดยสรุปแล้ว เส้นโค้งที่มีกลุ่มย่อย ซึ่งมีอันดับเป็นจำนวนเฉพาะขนาด $2b$ บิต จะให้ระดับความปลอดภัยประมาณ b บิต

นี่คือที่มาหลักของประสิทธิภาพใน ECC สำหรับกลุ่มปัญหาที่เกี่ยวข้องกับ $\mathbb{Z}_p^*$ และการแยกตัวประกอบจำนวนเต็ม อัลกอริทึมในตระกูล index calculus และ number field sieve มีเวลาการทำงานในระดับ sub-exponential ด้วยเหตุนี้ ขนาดของกุญแจจึงจำเป็นต้องเพิ่มขึ้นในอัตราที่เร็วกว่าแบบเชิงเส้น เมื่อระดับความปลอดภัยสูงขึ้น ในขณะที่ยังไม่มีอัลกอริทึมในลักษณะเดียวกันนี้สำหรับเส้นโค้งวงรีทั่วไปบนฟิลด์จำนวนเฉพาะ ตารางที่ 2 แสดงให้เห็นถึงช่องว่างดังกล่าว โดยอ้างอิงตามเกณฑ์ความเท่าเทียมกันของขนาดกุญแจที่กำหนดโดย NIST[10]

ตารางที่ 2: ขนาดกุญแจที่ให้ความปลอดภัยเทียบเท่ากัน ตามเกณฑ์ของ NIST SP 800-57 โดยอัตราส่วนเปรียบเทียบขนาด RSA modulus กับขนาดกุญแจ ECC ที่เล็กที่สุดที่ให้ความปลอดภัยเท่ากัน

Security (bits)	Symmetric	RSA / DH modulus	ECC key size	Ratio
80	2TDEA	1024	160–223	6 : 1
112	3TDEA	2048	224–255	9 : 1
128	AES-128	3072	256–383	12 : 1
192	AES-192	7680	384–511	20 : 1
256	AES-256	15360	512+	30 : 1

2.2. Weak Curves

เส้นโค้งบางประเภทมีความเสี่ยงต่อการถูกโจมตีด้วยวิธีที่รวดเร็วกว่าปกติมาก จึงควรหลีกเลี่ยงการใช้งานได้แก่

- **เส้นโค้ง Supersingular และเส้นโค้งที่มี Embedding degree ต่ำ**
  การโจมตีแบบ MOV[11] จะใช้ Weil pairing หรือ Tate pairing เพื่อแปลงปัญหา ECDLP ไปสู่ปัญหา Discrete logarithm ในฟิลด์ขยาย $\mathbb{F}_{p^k}$ ซึ่งหากค่า Embedding degree k มีขนาดเล็ก จะสามารถใช้วิธี Index calculus ที่มีความซับซ้อนในระดับต่ำกว่าเอกซ์โพเนนเชียล ในการแก้ปัญหาได้ (ทั้งนี้เส้นโค้งประเภท Pairing-friendly ถูกออกแบบมาให้มีค่า k ขนาดเล็กโดยเจตนาเพื่อวัตถุประสงค์เฉพาะ โดยจะมีการกำหนดพารามิเตอร์อื่นให้มีขนาดใหญ่ขึ้น)
- **เส้นโค้ง Anomalous**
  หากจำนวนจุดบนเส้นโค้ง $\left|E\left(\mathbb{F}_p\right)\right|$ เท่ากับ p (นั่นคือค่า Trace $t = 1$) ปัญหา ECDLP จะสามารถแก้ไขได้ภายในเวลาที่เป็นพหุนาม โดยใช้เทคนิค p-adic lifting[12]
- **เส้นโค้งบนฟิลด์ขยายที่มีดีกรีเป็นจำนวนประกอบ**
  เส้นโค้งประเภทนี้มีความเสี่ยงต่อการถูกโจมตีแบบ Weil descent และ summation-polynomial
- **เส้นโค้งที่มีลำดับของกลุ่มเป็นจำนวนที่แยกตัวประกอบได้ง่าย**
  เส้นโค้งประเภทนี้สามารถถูกถอดรหัสได้ด้วยอัลกอริทึมของ Pohlig–Hellman

ในการกำหนดเส้นโค้งมาตรฐานจะมีการคัดเลือกโดยหลีกเลี่ยงลักษณะที่ไม่ปลอดภัยเหล่านี้ทั้งหมด นอกจากนี้ โครงการ SafeCurves[13] ยังได้ขยายเกณฑ์การพิจารณาเพิ่มเติม โดยครอบคลุมถึงความปลอดภัยในขั้นตอนการนำไปใช้งานจริง นอกเหนือไปจากความปลอดภัยของปัญหา ECDLP เพียงอย่างเดียว

3. Cryptographic Protocols

ในกรณีทั้งหมดต่อไปนี้ กุญแจส่วนตัวของผู้ใช้คือจำนวนเต็มสุ่ม $d \in [1, n-1]$ และกุญแจสาธารณะที่สอดคล้องกันคือจุด $Q = dG$

3.1. Elliptic Curve Diffie–Hellman (ECDH)

ECDH ช่วยให้คู่สนทนาสองฝ่ายสามารถสร้างความลับร่วมกันผ่านช่องทางสาธารณะได้ โดยกระบวนการแลกเปลี่ยนดังกล่าวแสดงไว้ในรูปที่ 3

Alice	**ช่องทางสาธารณะ**	**Bob**
pick random $a \in [1, n-1]$		pick random $b \in [1, n-1]$
$A = aG$	$A \longrightarrow$	$B = bG$
	$\longleftarrow B$	
$S = aB$		$S = bA$
$aB = a(bG) = (ab)G = b(aG) = bA$		
session key $= \mathbf{KDF}(x_S)$		

รูปที่ 3: การตกลงกุญแจร่วมกันแบบ Elliptic Curve Diffie–Hellman

ผู้ดักฟังจะเห็นค่า G, A และ B จะต้องคำนวณหาค่า abG ให้ได้ ปัญหานี้เรียกว่าปัญหาการคำนวณแบบดิฟฟี-เฮลล์แมนซึ่งเชื่อกันว่ามีความยากเทียบเท่ากับปัญหาลอการิทึมไม่ต่อเนื่องบนเส้นโค้งวงรี สำหรับเส้นโค้งมาตรฐาน โดยจุดที่ได้ร่วมกันนั้นจะไม่มีการนำไปใช้งานโดยตรง แต่จะนำค่าพิกัด x ของจุดดังกล่าวไปผ่านฟังก์ชั่นสร้างกุญแจ

ในกลุ่มตัวอย่างนี้ หาก Alice เลือก $a = 3$ จะได้ $A = (10, 6)$ และหาก Bob เลือก $b = 7$ จะได้ $B = (0, 6)$ ทั้งสองฝ่ายจะได้ค่า $S = 21G = 2G = (6, 3)$

เนื่องจาก ECDH แบบปกติไม่มีกลไกการยืนยันตัวตน ผู้โจมตีแบบเชิงรุกจึงอาจแทรกตัวอยู่ระหว่าง Alice และ Bob ได้ ในทางปฏิบัติ ค่าสาธารณะจะถูกลงนามกำกับไว้ (เช่น ในโปรโตคอล TLS) หรือผูกไว้กับกุญแจระบุตัวตนระยะยาว (เช่น ในโปรโตคอล X3DH ของ Signal) การใช้กุญแจชั่วคราวที่สร้างขึ้นใหม่สำหรับทุกเซสชันจะช่วยให้เกิดคุณสมบัติความลับแบบส่งต่อ ซึ่งหมายความว่าหากกุญแจระยะยาวถูกบุกรุกในภายหลัง ข้อมูลของเซสชันที่ผ่านมาก็จะไม่ถูกเปิดเผย ทั้งนี้ โปรโตคอล TLS 1.3 กำหนดให้ต้องมีการแลกเปลี่ยนกุญแจชั่วคราวสำหรับการสร้างการเชื่อมต่อทุกครั้งที่ไม่ได้เป็นการกลับมาเชื่อมต่อใหม่[5]

3.2. Elliptic Curve Digital Signature Algorithm (ECDSA)

ECDSA ถูกกำหนดมาตรฐานไว้ใน FIPS 186[14] เป็นอัลกอริทึมที่พัฒนาต่อยอดมาจาก DSA โดยใช้หลักการของเส้นโค้งวงรี กำหนดให้ H เป็นฟังก์ชั่นแฮชเชิงรหัสลับ และให้ z คือค่าที่ได้จากบิตทางซ้ายสุดจำนวน $\lceil \log_2 n \rceil$ บิต ของผลลัพธ์จากฟังก์ชั่นแฮช $H(m)$ สำหรับข้อความ m ใดๆ

อัลกอริทึมที่ 2: การสร้างและการตรวจสอบลายเซ็นดิจิทัลแบบ ECDSA

การสร้างลายเซ็น (d, m):
1. เลือกค่าสุ่ม $k \in [1, n-1]$ ขึ้นมาใหม่ โดยให้เป็นค่าความลับและสุ่มแบบกระจายตัวสม่ำเสมอ
2. คำนวณหา $(x_1, y_1) = kG$ และกำหนดให้ $r = x_1 \bmod n$ หาก $r = 0$ ให้ย้อนกลับไปทำขั้นตอนที่ 1 ใหม่
3. คำนวณหา $s = k^{-1}(z + rd) \bmod n$ หาก $s = 0$ ให้ย้อนกลับไปทำขั้นตอนที่ 1 ใหม่
4. ส่งผลลัพธ์เป็นลายเซ็น (r, s)

การตรวจสอบลายเซ็น $(Q, m, (r, s))$:
1. ตรวจสอบว่า $r, s \in [1, n-1]$ และ Q เป็นจุดที่ถูกต้องบนเส้นโค้งซึ่งมีอันดับ (order) เท่ากับ n
2. คำนวณหา $w = s^{-1} \bmod n$, $u_1 = zw \bmod n$ และ $u_2 = rw \bmod n$
3. คำนวณหา $(x_1, y_1) = u_1 G + u_2 Q$ หากผลลัพธ์ที่ได้คือจุดอนันต์ ($\mathcal{O}$) ให้ปฏิเสธลายเซ็นนี้
4. ยอมรับลายเซ็นก็ต่อเมื่อ $r \equiv x_1 \pmod n$ เท่านั้น

ความถูกต้องเป็นผลมาจาก $u_1 G + u_2 Q = s^{-1}(z + rd)G = kG$

ค่า nonce จะต้องถูกเก็บเป็นความลับและห้ามนำกลับมาใช้ซ้ำเด็ดขาด หากลายเซ็นสองลายเซ็น (r, s_1) และ (r, s_2) ในข้อความที่แตกต่างกันใช้ nonce k ร่วมกัน (ซึ่งมองเห็นได้เป็น r ที่เท่ากัน) ใครก็ตามก็สามารถกู้คืนคีย์ส่วนตัวได้:

$$k = \frac{z_1 - z_2}{s_1 - s_2} \bmod n, \qquad d = \frac{s_1 k - z_1}{r} \bmod n \tag{7}$$

นี่คือวิธีเดียวกับที่คีย์สำหรับลงนามโค้ด ของ PlayStation 3 ถูกดึงออกมาในปี ค.ศ. 2010[15] โดยในการใช้งานนั้นมีการใช้ค่าคงที่ k ซึ่งเพียงแค่บิตบางส่วนของค่า nonce ที่รั่วไหลออกมา ก็เพียงพอต่อการกู้คืนคีย์ด้วยเทคนิคที่อาศัยโครงสร้างแบบแลตทิซ (lattice-based key recovery)[16] ทั้งนี้ มาตรฐาน RFC 6979[17] ได้ขจัดปัญหาการต้องพึ่งพาตัวสร้างตัวเลขสุ่ม โดยใช้วิธีคำนวณค่า k แบบกำหนดผลลัพธ์ตายตัวจากค่า d และ $H(m)$ แทน

3.3. EdDSA and Ed25519

EdDSA[18][19] เป็นรูปแบบลายเซ็นดิจิทัลในสไตล์ Schnorr ซึ่งสร้างขึ้นบนเส้นโค้ง Edwards แบบบิดเบี้ยว โดยได้รับการออกแบบมาเพื่อขจัดข้อบกพร่องต่าง ๆ ที่พบใน ECDSA สำหรับ Ed25519 ซึ่งเป็นรูปแบบที่รู้จักกันมากที่สุดของ EdDSA นั้น มีกระบวนการทำงานดังนี้: เริ่มต้นด้วยการนำค่าตั้งต้นที่เป็นความลับ (secret seed) ขนาด 32 ไบต์ มาผ่านฟังก์ชั่นแฮช SHA-512 ผลลัพธ์ที่ได้จะถูกแบ่งออกเป็นสองส่วน ส่วนแรกใช้เป็นค่าสเกลาร์ลับ a โดยที่กุญแจสาธารณะคือ $A = aB$ (เมื่อ B คือจุดฐาน หรือ base point) ส่วนอีกครึ่งหนึ่งจะถูกใช้เป็นค่าคำนำหน้าลับสำหรับการลงลายเซ็นในข้อความ M มีขั้นตอนดังนี้:

$$r = H(\text{prefix} \parallel M), \quad R = rB, \quad S = (r + H(R \parallel A \parallel M) \cdot a) \bmod \ell, \tag{8}$$

โดยที่ ℓ คืออันดับของกลุ่มที่เป็นจำนวนเฉพาะ ลายเซ็นคือ (R, S) ผู้ตรวจสอบจะทำการตรวจสอบว่า $8SB = 8R + 8H(R \parallel A \parallel M)A$

เมื่อเปรียบเทียบกับ ECDSA แล้ว EdDSA มีข้อได้เปรียบในทางปฏิบัติหลายประการ:

- nonce เป็นแบบกำหนดได้ ดังนั้นเครื่องกำเนิดเลขสุ่มที่อ่อนแอจึงไม่สามารถรั่วไหลคีย์ได้
- การลงนามไม่จำเป็นต้องใช้การผกผันโมดูลาร์
- กฎการบวกของ Edwards ที่สมบูรณ์ไม่มีกรณีพิเศษ ซึ่งทำให้โค้ดเวลาคงที่ตรงไปตรงมา
- การตรวจสอบลายเซ็นจำนวนมากสามารถรวมกลุ่มกันเพื่อเพิ่มความเร็ว

Bitcoin ได้นำลายเซ็น BIP 340 Schnorr ที่เกี่ยวข้องอย่างใกล้ชิดมาใช้แทน secp256k1 ในการอัปเกรด Taproot ในปี 2021[20]

3.4. Elliptic Curve Integrated Encryption Scheme (ECIES)

โดยทั่วไปแล้ว ECC ไม่ค่อยถูกนำมาใช้ในการเข้ารหัสข้อมูลโดยตรง แต่จะใช้วิธีการเข้ารหัสแบบผสม เช่น ECIES ซึ่งเป็นการรวมกระบวนการแลกเปลี่ยนกุญแจแบบ ECDH ชั่วคราวเข้ากับการเข้ารหัสแบบสมมาตร โดยผู้ส่งจะสร้างคู่กุญแจชั่วคราวขึ้นมา $(e, E = eG)$ และคำนวณค่า $S = eQ_{\text{recipient}}$ จากนั้นนำค่า S ไปผ่านฟังก์ชันสร้างกุญแจเพื่อให้ได้กุญแจสำหรับการเข้ารหัสและกุญแจสำหรับ MAC แล้วจึงส่งค่า E ไปพร้อมกับข้อมูลที่เข้ารหัสซึ่งมีการตรวจสอบความถูกต้อง ทางด้านผู้รับจะคำนวณค่า S ใหม่ด้วยสูตร $S = dE$ รูปแบบเดียวกันนี้ยังเป็นพื้นฐานของ HPKE (ตามมาตรฐาน RFC 9180) ซึ่งถูกนำไปใช้ใน TLS Encrypted Client Hello และ Messaging Layer Security อีกด้วย

4. Standardized Curves

การจะสร้างพารามิเตอร์ของเส้นโค้งให้มีความปลอดภัยนั้น มีรายละเอียดที่ซับซ้อนและละเอียดมาก ดังนั้นในความเป็นจริงระบบส่วนใหญ่จึงเลือก “เส้นโค้งมาตรฐาน” ที่ผ่านการตรวจสอบมาอย่างดีแล้วเพียงไม่กี่แบบมาใช้งาน ตารางด้านล่างนี้สรุปเส้นโค้งที่นำไปใช้แพร่หลายจนถึงปัจจุบัน

ตารางที่ 3: เส้นโค้งมาตรฐานที่นำไปใช้งานอย่างแพร่หลาย

ชื่อเส้นโค้ง	**รูปแบบสมการ**	**Bit**	**ระดับความปลอดภัย**	**ตัวอย่างการนำไปใช้งาน**
P-256 (secp256r1)	Weierstrass	256	128	TLS, FIDO2/WebAuthn, สมาร์ตการ์ด, บัตรประชาชน
P-384	Weierstrass	384	192	ระบบของหน่วยงานรัฐบาล (มาตรฐาน CNSA 1.0)
secp256k1	Koblitz	256	128	ระบบคริปโทเคอร์เรนซี เช่น Bitcoin, Ethereum
brainpoolP256r1	Weierstrass	256	128	บัตร eID ของยุโรป และหน่วยงานรัฐ
Curve25519	Montgomery	255	~128	โปรโตคอล TLS 1.3, SSH, แอป Signal, WireGuard
Ed25519	Twisted Edwards	255	~128	คีย์ SSH, การเซ็นรับรองซอฟต์แวร์, เครือข่าย Tor
Curve448 / Ed448	Montgomery	448	~224	ระบบ TLS และลายเซ็นดิจิทัลที่ต้องการความปลอดภัยสูงมาก

4.1. รูปแบบของเส้นโค้ง

กลุ่มเดียวกันสามารถเขียนแทนได้ด้วยสมการที่แตกต่างกัน ซึ่งการเลือกใช้สมการนั้นส่งผลต่อทั้งความเร็วและความปลอดภัย

- **Short Weierstrass** แบบสั้น $y^2 = x^3 + ax + b$ เป็นรูปแบบทั่วไป ใช้เส้นโค้งของ NIST, SEC และ Brainpool สูตรการบวกมีกรณียกเว้น
- **Montgomery** คำนวณที่ทำงานได้รวดเร็ว $By^2 = x^3 + Ax^2 + x$ ใช้เวลาได้คงที่ โดนสมบัตินี้ทำให้ต้านด้วยการถูกแฮกด้วยกับจับเวลา เช่น X25519, X448

- **Twisted Edwards** $ax^2 + y^2 = 1 + dx^2y^2$ กฎการบวกสมบูรณ์ที่ไม่มีกรณียกเว้น และเป็นรูปแบบที่เร็วที่สุด (Ed25519, Ed448)

เพื่อหลีกเลี่ยงการผกผันในฟิลด์ที่มีต้นทุนสูง การใช้งานจึงใช้พิกัดเชิงโปรเจกทีฟหรือพิกัดจาโคเบียนด้วยตัวอย่างเช่น สามพิกัดจาโคเบียน $(X : Y : Z)$ แสดงถึงจุดแอฟฟีน $(X/Z^2, Y/Z^3)$ ดังนั้นการคูณสเกลาร์จึงต้องการการผกผันเพียงครั้งเดียวในตอนท้าย

4.2. ความเชื่อมั่นในการสร้างเส้นโค้ง

เส้นโค้งของ NIST ถูกเผยแพร่ในปี ค.ศ. 1999 โดยมีที่มาจากค่าเริ่มต้น ซึ่งไม่เคยมีการอธิบายถึงแหล่งกำเนิดที่แน่ชัด หลังจากมีการเปิดเผยข้อมูลในปี ค.ศ. 2013 เกี่ยวกับอัลกอรีทึมสร้างตัวเลขสุ่ม Dual_EC_DRBG ที่มีการฝังช่องโหว่ไว้ ผู้คนจำนวนมากในวงการนี้จึงออกมาตั้งคำถามว่า ค่าคงที่ที่ไม่มีการอธิบายที่มาเหล่านี้อาจแฝงจุดอ่อนเอาไว้หรือไม่ แม้ว่าจะยังไม่มีการค้นพบการโจมตีใดๆ ต่อเส้นโค้ง P-256 และเส้นโค้งดังกล่าวยังคงได้รับการรับรองและยังใช้งานอย่างแพร่หลาย แต่เหตุการณ์ดังกล่าวก็เกิดการตั้งคำถาม และการต้องการความโปร่งใส ที่สามารถตรวจสอบได้ชัดเจนมากยิ่งขึ้น เช่น Curve25519 และ Brainpool[21] ซึ่งพารามิเตอร์ต่างๆ ถูกกำหนดขึ้นตามกฎเกณฑ์ที่โปร่งใส ทั้งนี้ในปี ค.ศ. 2023 NIST ได้เพิ่ม Curve25519 และ Curve448 เข้าไว้ในข้อแนะนำใน SP 800-186[6] และ FIPS 186-5[14]

5. Implementation Security

ทฤษฎีคณิตศาสตร์พื้นฐานของ ECC มีระบบรักษาความปลอดภัยที่แข็งแรงมาตลอด การที่ระบบคริปโตถูกแฮกแทบจะเกิดจากความผิดพลาดในขั้นตอน “การนำไปเขียนโค้ดหรือสร้างระบบจริง” ทั้งสิ้น โดยข้อผิดพลาดที่เจอบ่อยมีดังนี้

- **โดนจับผิดพฤติกรรมการทำงาน**

หากโค้ดที่เราเขียนมามีการใช้คำสั่ง หรือใช้เวลาคำนวณช้า-เร็วไม่เท่ากัน แฮกเกอร์สามารถเดา “Private key” ได้จากการจับเวลาที่คอมพิวเตอร์ใช้ประมวลผล แอบวัดอัตราการกินไฟ หรือดูดคลื่นแม่เหล็กไฟฟ้าที่แผ่ออกมา

วิธีแก้: ต้องเขียนโค้ดแบบใช้เวลาทำงานเท่ากันเสมอ รวมถึงใช้เทคนิคทางคณิตศาสตร์ เช่น Montgomery ladder และการสุ่มเพื่อพรางข้อมูลไม่ให้เดาได้

- **โดนหลอกด้วยข้อมูลนอกเส้นโค้ง**

ถ้าเราเขียนโปรแกรมรับข้อมูลเข้ามา โดยไม่ตรวจเช็กให้ดีว่า “จุด” ที่รับเข้ามาอยู่บนเส้นโค้งจริง ๆ หรือไม่ แฮกเกอร์อาจจะจงใจส่งข้อมูลของเส้นโค้งอื่นที่อ่อนแอกว่ามาให้ระบบคำนวณ ซึ่งทำให้แฮกเกอร์แงะ Private key ได้ทีละส่วน (ระบบ TLS เคยโดนเจาะด้วยวิธีนี้มาแล้ว[22])

วิธีแก้: ต้องตรวจสอบทุกข้อมูลที่รับเข้ามาเสมอ หรือเปลี่ยนไปใช้เส้นโค้งที่มีการออกแบบมาเพื่อป้องกันเรื่องนี้แต่แรก อย่างเช่น Curve25519

- **ระบบสุ่มเลขมีปัญหาตอนเซ็นชื่อ**

ในการสร้างลายเซ็นดิจิทัล ระบบจะต้องใช้ตัวเลขสุ่มที่เรียกว่า Nonce หากระบบสุ่มเลขออกมาซ้ำกันเดาทางได้ หรือสุ่มไม่ดี Private key ของเราจะถูกเปิดเผยให้แฮกเกอร์รู้

วิธีแก้: เลิกพึ่งพาระบบสุ่มเลขในจังหวะที่ทำการเซ็น แต่เปลี่ยนไปใช้วิธีสร้าง Nonce แบบคำนวณตายตัวตามมาตรฐาน เช่น RFC 6979[17] หรือ EdDSA[19] แทน

- **ความสับสนเรื่องรูปแบบข้อมูล**

เส้นโค้งบางแบบยอมให้ข้อมูลลายเซ็นหรือจุด “ตัวเดียวกัน” สามารถเขียนออกมาในรูปแบบที่หน้าตาต่างกันได้ เช่น คำว่า “รถยนต์” กับ “รถ” ที่ความหมายเดียวกันแต่เขียนต่างกัน สิ่งนี้ทำให้ระบบที่คิดเอาเองว่ารูปแบบข้อมูลต้องมีแค่อย่างเดียว ทำงานรวนหรือเกิดบั๊กได้

วิธีแก้: ใช้เครื่องมือช่วย เพื่อบังคับให้ข้อมูลอ่านได้รูปแบบเดียวอย่างชัดเจน เช่น Ristretto255

- **แกล้งทำให้ฮาร์ดแวร์ทำงานรวน**

แฮกเกอร์อาจโจมตีทางฮาร์ดแวร์ เช่น แกล้งกระตุกกระแสไฟขณะที่อุปกรณ์กำลังคำนวณรหัสผ่านเพื่อทำให้ฮาร์ดแวร์ทำงานเพี้ยนจนรหัสหลุดออก

วิธีแก้: อุปกรณ์อย่างสมาร์ตการ์ดจะต้องออกแบบให้ตรวจสอบผลลัพธ์การคำนวณของตัวเองก่อนส่งข้อมูลออกไป

การเขียนโค้ดระบบ ECC ให้ปลอดภัยนั้นมีรายละเอียดจุกจิกที่ต้องระวังเยอะมาก ข้อแนะนำคือ “อย่าเขียนโค้ดเข้ารหัสขึ้นมาใช้งานเอง” แต่ควรไปหยิบเอาไลบรารีที่สร้างไว้แล้วและถูกตรวจสอบแล้วมาใช้งานแทนจะปลอดภัยกว่ามาก

6. The Quantum Threat and Post-Quantum Migration

อัลกอริทึมของ Shor[23] แก้ปัญหาทั้งการแยกตัวประกอบและลอการิทึมไม่ต่อเนื่องในกลุ่มอาบีเลียนจำกัดใด ๆ ได้ในเวลาพหุนามบนควอนตัมคอมพิวเตอร์ที่ทนต่อข้อผิดพลาดที่มีขนาดใหญ่พอสมควร ซึ่งรวมถึง ECDLP ด้วย ดังนั้นควอนตัมคอมพิวเตอร์ที่เกี่ยวข้องกับการเข้ารหัส หรือ CRQC จึงจะทำลาย ECDH, ECDSA และ EdDSA ได้อย่างสมบูรณ์ไม่ว่าขนาดคีย์จะเป็นเท่าใด

เนื่องจากคีย์ ECC มีขนาดเล็ก จึงอาจถูกควอนตัมคอมพิวเตอร์ถอดรหัสได้ก่อน RSA ในระดับความปลอดภัยแบบดั้งเดิมที่เทียบเคียงได้ Roetteler et al.[24] ประมาณการว่าการคำนวณลอการิทึมไม่ต่อเนื่องบน P-256 ต้องการคิวบิตเชิงตรรกะประมาณ $9n + 2\lceil \log_2 n \rceil + 10 \approx 2330$ คิวบิต ซึ่งน้อยกว่าที่ประมาณสำหรับแยกตัวประกอบ RSA-3072 แม้ว่าวงจรจะยังคงลึกมากก็ตาม อุปกรณ์ในปัจจุบันยังห่างจากระดับนี้หลายเท่าตัว แต่ความเสี่ยงของการเก็บข้อมูลตอนนี้แล้วถอดรหัสภายหลังนั้นก็เป็นเรื่องจริง การรับส่งข้อมูลที่เข้ารหัสซึ่งบันทึกไว้ในปัจจุบันอาจจะถูกถอดรหัสได้เมื่อมี CRQC ดังนั้นการแลกเปลี่ยนคีย์จะต้องถูกย้ายไปก่อนที่จะมีการสร้าง CRQC ขึ้นมา

NIST ได้เผยแพร่มาตรฐานการเข้ารหัสลับหลังยุคควอนตัมฉบับแรกในเดือนสิงหาคม ค.ศ. 2024 ได้แก่ ML-KEM ตามมาตรฐาน FIPS 203 สำหรับการห่อหุ้มคีย์[25] และ ML-DSA ตามมาตรฐาน FIPS 204 กับ SLH-DSA สำหรับลายเซ็นดิจิทัล[26] ส่วนแผนการเปลี่ยนผ่านฉบับร่างของ NIST เสนอให้เลิกใช้อัลกอริทึมที่เสี่ยงต่อควอนตัม เช่น ECDH และ ECDSA หลังปี ค.ศ. 2030 และไม่อนุญาตให้ใช้หลังปี ค.ศ. 2035[27]

กลยุทธ์การโยกย้ายที่แพร่หลายคือการเข้ารหัสลับแบบไฮบริด โดยให้อัลกอริทึมแบบดั้งเดิมและแบบหลังยุคควอนตัมทำงานร่วมกัน และเซสชันจะปลอดภัยตราบใดที่อัลกอริทึมตัวใดตัวหนึ่งยังไม่ถูกทำลาย การแลกเปลี่ยนคีย์แบบไฮบริด X25519MLKEM768 ถูกเปิดใช้งานโดยค่าเริ่มต้นในเบราว์เซอร์หลักและ CDN อยู่แล้ว ดังนั้น ECC ยังคงเป็นส่วนสำคัญของระบบที่ใช้งานอยู่ระหว่างช่วงเปลี่ยนผ่าน โดยปัจจุบันทำงานร่วมกับแผนการที่ใช้แลตทิซ แทนที่จะถูกแทนที่ในชั่วข้ามคืน

7. Conclusion

Elliptic Curve Cryptography (ECC) วิทยาการเข้ารหัสด้วยเส้นโค้งวงรี เปลี่ยนกฎเรขาคณิตอย่างง่าย โดยการบวกจุดบนเส้นเชื่อมและเส้นสัมผัสของจุดต่าง ๆ ให้เป็นกลุ่มก้อนที่ปัญหาลอการิทึมแบบไม่ต่อเนื่อง สามารถต้านทานการโจมตีแบบดั้งเดิมได้ทั้งหมด ยกเว้นอัลกอริทึมรากที่สองทั่วไป ทำให้เป็นข้อได้เปรียบที่สำคัญของ ECC คือเรื่องของความปลอดภัย ประมาณ 128 บิตจากคีย์ 256 บิตที่กระทัดรัดและรวดเร็ว คุณสมบัติเหล่านี้ทำให้ ECC กลายเป็นเทคโนโลยีคีย์สาธารณะพื้นฐานที่สำคัญสำหรับอินเทอร์เน็ต, อุปกรณ์เคลื่อนที่และฝังตัว รวมถึงสกุลเงินดิจิทัล

จากประวัติศาสตร์ของ ECC ยังช่วยบอกได้ว่า คณิตศาสตร์ที่แข็งแกร่งนั้นยังคงจำเป็นแต่ก็ไม่เพียงพอ เส้นโค้งที่อ่อนแอ, อินพุตที่ไม่ได้รับการตรวจสอบ, ช่องทางข้างและการใช้โนนซ์ผิดวิธี ทำให้เกิดช่องโหว่มากกว่าความก้าวหน้าใด ๆ ในการต่อต้าน ECDLP การออกแบบสมัยใหม่ เช่น Curve25519 และ Ed25519 ตอบสนองด้วยการทำให้การสร้างระบบที่ปลอดภัยให้ง่ายที่สุด อนาคตการคำนวณควอนตัมยังทำให้ ECC อาจเริ่มมีอายุการใช้งานที่จำกัดลง อย่างไรก็ตามอย่างน้อยในอีกทศวรรษข้างหน้า ECC ก็จะยังคงมีการใช้งานได้ต่อไป โดยจะมีบทบาทมากขึ้นในฐานะครึ่งหนึ่งของโครงสร้างแบบไฮบริดที่ผสมการเข้ารหัสลับดั้งเดิมกับแบบหลังยุคควอนตัม

References

[1] W. Diffie และ M. E. Hellman, “New Directions in Cryptography”, *IEEE Transactions on Information Theory*, ปี 22, ฉบับที่ 6, น. 644–654, 1976, doi: 10.1109/TIT.1976.1055638.

[2] R. L. Rivest, A. Shamir, และ L. Adleman, “A Method for Obtaining Digital Signatures and Public-Key Cryptosystems”, *Communications of the ACM*, ปี 21, ฉบับที่ 2, น. 120–126, 1978, doi: 10.1145/359340.359342.

[3] N. Koblitz, “Elliptic Curve Cryptosystems”, *Mathematics of Computation*, ปี 48, ฉบับที่ 177, น. 203–209, 1987, doi: 10.1090/S0025-5718-1987-0866109-5.

[4] V. S. Miller, “Use of Elliptic Curves in Cryptography”, ใน *Advances in Cryptology – CRYPTO '85*, ใน Lecture Notes in Computer Science, vol. 218. Springer, 1986, น. 417–426. doi: 10.1007/3-540-39799-X_31.

[5] E. Rescorla, “The Transport Layer Security (TLS) Protocol Version 1.3”, Internet Engineering Task Force (IETF), technical report RFC 8446, ส.ค. 2018. [ออนไลน์]. เข้าถึงได้จาก: https://www.rfc-editor.org/rfc/rfc8446

[6] L. Chen, D. Moody, A. Regenscheid, A. Robinson, และ K. Randall, “Recommendations for Discrete Logarithm-Based Cryptography: Elliptic Curve Domain Parameters”, National Institute of Standards and Technology (NIST), technical report SP 800-186, ก.พ. 2023. doi: 10.6028/NIST.SP.800-186.

[7] L. C. Washington, *Elliptic Curves: Number Theory and Cryptography*, 2 พิมพ์ครั้งที่. Chapman & Hall/CRC, 2008. doi: 10.1201/9781420071474.

[8] P. L. Montgomery, “Speeding the Pollard and Elliptic Curve Methods of Factorization”, *Mathematics of Computation*, ปี 48, ฉบับที่ 177, น. 243–264, 1987, doi: 10.1090/S0025-5718-1987-0866113-7.

[9] J. M. Pollard, “Monte Carlo Methods for Index Computation (mod p)”, *Mathematics of Computation*, ปี 32, ฉบับที่ 143, น. 918–924, 1978, doi: 10.1090/S0025-5718-1978-0491431-9.

[10] E. Barker, “Recommendation for Key Management: Part 1 – General”, National Institute of Standards and Technology (NIST), technical report SP 800-57 Part 1 Rev. 5, พ.ค. 2020. doi: 10.6028/NIST.SP.800-57pt1r5.

[11] A. Menezes, T. Okamoto, และ S. A. Vanstone, “Reducing Elliptic Curve Logarithms to Logarithms in a Finite Field”, *IEEE Transactions on Information Theory*, ปี 39, ฉบับที่ 5, น. 1639–1646, 1993, doi: 10.1109/18.259647.

[12] N. P. Smart, “The Discrete Logarithm Problem on Elliptic Curves of Trace One”, *Journal of Cryptology*, ปี 12, ฉบับที่ 3, น. 193–196, 1999, doi: 10.1007/s001459900052.

[13] D. J. Bernstein และ T. Lange, “SafeCurves: Choosing Safe Curves for Elliptic-Curve Cryptography”. สืบค้น: 8 พฤศจิกายน 2026. [ออนไลน์]. เข้าถึงได้จาก: https://safecurves.cr.yp.to/

[14] National Institute of Standards and Technology, “Digital Signature Standard (DSS)”, technical report FIPS 186-5, ก.พ. 2023. doi: 10.6028/NIST.FIPS.186-5.

[15] fail0verflow, “Console Hacking 2010: PS3 Epic Fail”. ธันวาคม 2010.

[16] N. A. Howgrave-Graham และ N. P. Smart, “Lattice Attacks on Digital Signature Schemes”, *Designs, Codes and Cryptography*, ปี 23, ฉบับที่ 3, น. 283–290, 2001, doi: 10.1023/A:1011214926272.

[17] T. Pornin, “Deterministic Usage of the Digital Signature Algorithm (DSA) and Elliptic Curve Digital Signature Algorithm (ECDSA)”, Internet Engineering Task Force (IETF), technical report RFC 6979, ส.ค. 2013. [ออนไลน์]. เข้าถึงได้จาก: https://www.rfc-editor.org/rfc/rfc6979

[18] D. J. Bernstein, N. Duif, T. Lange, P. Schwabe, และ B.-Y. Yang, “High-Speed High-Security Signatures”, *Journal of Cryptographic Engineering*, ปี 2, ฉบับที่ 2, น. 77–89, 2012, doi: 10.1007/s13389-012-0027-1.

[19] S. Josefsson และ I. Liusvaara, “Edwards-Curve Digital Signature Algorithm (EdDSA)”, Internet Engineering Task Force (IETF), technical report RFC 8032, ม.ค. 2017. [ออนไลน์]. เข้าถึงได้จาก: https://www.rfc-editor.org/rfc/rfc8032

[20] P. Wuille, J. Nick, และ T. Ruffing, “Schnorr Signatures for secp256k1”, Bitcoin Improvement Proposals, technical report BIP 340, 2020. [ออนไลน์]. เข้าถึงได้จาก: https://github.com/bitcoin/bips/blob/master/bip-0340.mediawiki

[21] M. Lochter และ J. Merkle, “Elliptic Curve Cryptography (ECC) Brainpool Standard Curves and Curve Generation”, Internet Engineering Task Force (IETF), technical report RFC 5639, มี.ค. 2010. [ออนไลน์]. เข้าถึงได้จาก: https://www.rfc-editor.org/rfc/rfc5639

[22] T. Jager, J. Schwenk, และ J. Somorovsky, “Practical Invalid Curve Attacks on TLS-ECDH”, ใน *Computer Security – ESORICS 2015*, ใน Lecture Notes in Computer Science, vol. 9326. Springer, 2015, น. 407–425. doi: 10.1007/978-3-319-24174-6_21.

[23] P. W. Shor, “Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer”, *SIAM Journal on Computing*, ปี 26, ฉบับที่ 5, น. 1484–1509, 1997, doi: 10.1137/S0097539795293172.

[24] M. Roetteler, M. Naehrig, K. M. Svore, และ K. Lauter, “Quantum Resource Estimates for Computing Elliptic Curve Discrete Logarithms”, ใน *Advances in Cryptology – ASIACRYPT 2017*, ใน Lecture Notes in Computer Science, vol. 10625. Springer, 2017, น. 241–270. doi: 10.1007/978-3-319-70697-9_9.

[25] National Institute of Standards and Technology, “Module-Lattice-Based Key-Encapsulation Mechanism Standard”, technical report FIPS 203, ส.ค. 2024. doi: 10.6028/NIST.FIPS.203.

[26] National Institute of Standards and Technology, “Module-Lattice-Based Digital Signature Standard”, technical report FIPS 204, ส.ค. 2024. doi: 10.6028/NIST.FIPS.204.

[27] D. Moody, R. Perlner, A. Regenscheid, A. Robinson, และ D. Cooper, “Transition to Post-Quantum Cryptography Standards”, National Institute of Standards and Technology (NIST), technical report NIST IR 8547 (Initial Public Draft), พ.ย. 2024. doi: 10.6028/NIST.IR.8547.ipd.